Recuperación de video en vivo bajo pérdida de paquetes: I-headers entregados por el edge sobre SRT

Vicente Andrade Ercilla
Universidad Técnica Federico Santa María, Chile
IPD438 Seminario de Redes de Computadores

RESUMEN

En video en vivo con GoP (*Group of Pictures*) largo, un paquete perdido daña la imagen hasta el próximo cuadro intra, que puede llegar segundos después. SRT (*Secure Reliable Transport*) retransmite lo perdido, pero solo dentro de una latencia fija; lo que no llega a tiempo se descarta y el decodificador muestra bloques corruptos que se propagan. Proponemos que el *edge* (el servidor de borde que reparte el video a los espectadores) entregue un I-header (la misma imagen codificada además como intra, que ya viaja dentro de una SEI, *Supplemental Enhancement Information*) solo al espectador que perdió paquetes, sin pedir nada al codificador. Mostramos que el edge puede saber qué paquetes descartó el espectador usando solo la marca de tiempo de los ACK (*acknowledgement*) de SRT, sin cambiar el protocolo. Con un emulador en C++, cuyo núcleo está pensado para el microcontrolador STM32N6, comparamos UDP (*User Datagram Protocol*) y SRT con y sin I-header. Con cortes de 200 y 600 ms y $L = 120$ ms, SRT+I-header reduce los frames dañados en un 52 % frente a SRT solo, sin agregar latencia y con menos de 1 % más de tasa hacia el espectador. Con cortes de 20 ms, SRT solo ya basta. Sin pérdida, el espectador recibe exactamente el mismo stream.

Palabras clave: H.264, SRT, pérdida de paquetes, edge, GoP, PSNR.

I. INTRODUCCIÓN

En la transmisión de video en vivo, el codificador elige cada cuántos frames pone un cuadro intra (I), que se decodifica solo. Entre dos intra van cuadros predichos (P), que solo describen lo que cambió respecto del frame anterior. El grupo de cuadros entre dos intra se llama GoP (*Group of Pictures*). Un GoP largo ahorra bits, porque un intra pesa muchas veces lo que un P. A cambio, deja pocos puntos de partida: si se pierde un paquete, el error se copia de un P al siguiente hasta el próximo intra. Con GoP 90 a 30 fps, eso puede durar hasta 3 s.

SRT (*Secure Reliable Transport*) [1] es un protocolo abierto sobre UDP (*User Datagram Protocol*) muy usado para llevar video en vivo. Retransmite lo que el receptor reporta como perdido y entrega cada paquete con una latencia constante. Por eso es razonable preguntarse si con SRT todavía hay pérdidas que lleguen al decodificador. La respuesta es sí: SRT es confiable solo dentro de su latencia. Si un corte de la red dura más que esa latencia, SRT descarta el paquete y el espectador ve bloques corruptos (*macroblocking*), que se arrastran hasta el próximo intra (Sección IV).

Hay dos salidas conocidas. Subir la latencia retrasa la imagen de todos los espectadores. Acortar el GoP hace que todos paguen más bits. La idea de este trabajo es que el mecanismo para recuperarse ya está en la red, ya está pagado y hoy se descarta. AGiLE [2] envía hacia el *edge*, el servidor de borde que reparte el video a los espectadores, un I-header: una copia intra de un P, dentro de una SEI (*Supplemental Enhancement Information*). Lo usa solo para que quien entra al canal no espere el próximo intra. Nosotros usamos ese mismo I-header para recuperar pérdidas.

Objetivo. Que un espectador que pierde paquetes recupere la imagen en el próximo I-header, y no en el próximo intra, sin enviar un pedido al codificador por cada pérdida, y que el espectador sin pérdidas reciba exactamente lo mismo que con GoP largo. Las contribuciones son tres:

1. El edge le entrega el I-header solo al espectador que perdió paquetes (Sección V).
2. Una regla para que el emisor SRT (el edge) sepa qué paquetes descartó el receptor, solo con la marca de tiempo de los ACK (*acknowledgement*), sin cambiar el protocolo (Sección A).
3. Un emulador en C++ cuyo núcleo no usa memoria dinámica ni depende del sistema operativo, para reusarlo en el STM32N6, y una primera comparación de SRT con y sin I-header (Sección VI).

La Sección II resume lo necesario de H.264 y SRT, y la Sección III los trabajos relacionados. La Sección IV plantea el problema, la Sección V la estrategia y la arquitectura, la Sección VI los resultados y la Sección VII las conclusiones.

II. CONTENIDOS PREVIOS

A. H.264: intra, predicho, IDR e I-header

En H.264 [3], la imagen se divide en macrobloques de 16×16 píxeles, que se agrupan en *slices*. Cada slice va en una NAL (*Network Abstraction Layer unit*). Un P copia, para cada macrobloque, un bloque del frame anterior desplazado según un vector de movimiento, y le suma una corrección. Si el frame anterior está dañado, la copia también lo está. Un IDR (*Instantaneous Decoding Refresh*) es un intra que además vacía las referencias del decodificador (NAL tipo 5). Un intra no-IDR va como un slice normal (NAL tipo 1). El I-header de AGiLE [2]

es la misma imagen que un P, codificada además como intra no-IDR y escondida en una SEI (NAL tipo 6) de datos de usuario, que el decodificador ignora si no la conoce. La SEI *recovery point* le indica al decodificador que puede retomar desde un intra que no es IDR.

B. De la imagen a la red

Las NAL de un frame forman un PES (*Packetized Elementary Stream*) con su marca de tiempo. El TS (*Transport Stream* de MPEG, *Moving Picture Experts Group*) lo corta en paquetes de 188 B, y SRT lleva 7 de ellos (1316 B) en cada datagrama UDP, con número de secuencia y marca de tiempo. Si se pierde un paquete SRT, el slice que lo contiene se daña desde ese punto hasta su final. Con un slice por fila de macrobloques se sabe en qué frame y en qué fila quedó el daño.

C. SRT en modo live

El receptor de SRT [1] detecta un hueco en los números de secuencia y avisa de inmediato con un NAK (*negative acknowledgement*). Repite ese aviso cada $\max(20\,\text{ms}, (\text{RTT} + 4\,\text{RTTVar})/2)$, donde RTT es el tiempo de ida y vuelta y RTTVar su variación. El emisor retransmite lo pedido con prioridad. Cada 10 ms, el receptor envía un ACK con el primer número de secuencia que le falta. TSBPD (*Timestamp-Based Packet Delivery*) entrega cada paquete a la aplicación en su marca de tiempo más la latencia L, que por defecto es de 120 ms. Si al llegar ese momento un paquete todavía falta, TLPKTDROP (*Too-Late Packet Drop*) lo salta y el receptor envía un ACK que lo cubre como si hubiera llegado. Según el borrador, "que un paquete se saltó queda desconocido para el emisor" [1].

III. Trabajos relacionados

AGiLE [2] (ByteDance, 2024) usa un GoP de 10 s con un I-header cada 2 s, y el edge solo se lo entrega a quien entra al canal. Logra un BD-rate (*Bjøntegaard Delta rate*, la diferencia promedio de bits a igual calidad) de $-3{,}5$ % frente a un GoP fijo de 2 s, con $+2{,}8$ % de datos en total. Entrar por un I-header le cuesta 1,1 puntos de VMAF (*Video Multimethod Assessment Fusion*, Sección D), porque los P siguientes fueron calculados sobre el P y no sobre el I-header. AGiLE no trata las pérdidas.

Keyframe injection [4, 5] mantiene un flujo aparte de keyframes, del que pide uno el reproductor que entra o que pierde paquetes. La caída de calidad es menor que 0,94 puntos de VMAF [4]. El estudio se extendió a tres generaciones de estándares de video, desde H.264 [5]. Para no generar picos de bitrate, una versión posterior repara con un P que se apoya en un frame anterior a la pérdida [6]. Estos trabajos muestran que un keyframe extra repara la pérdida. Nuestra propuesta se diferencia en que no necesita un flujo aparte (el I-header ya viaja en el stream) ni un pedido del reproductor (decide el edge).

FrameTrace [7] mide telemetría por frame en Media over QUIC, con FRR (*Frame Render Rate*, frames mostrados sobre frames enviados). Con 1 % de pérdida, informa VMAF 72,6 con GoP de 0,5 s y 55,7 con GoP de 2 s. Su controlador acorta el GoP cuando hay pérdida, lo que requiere un lazo hasta el codificador. El PLI (*Picture Loss Indication*) de RTP (*Real-time Transport Protocol*) es el lazo clásico: el receptor avisa y el codificador manda un intra que reciben todos.

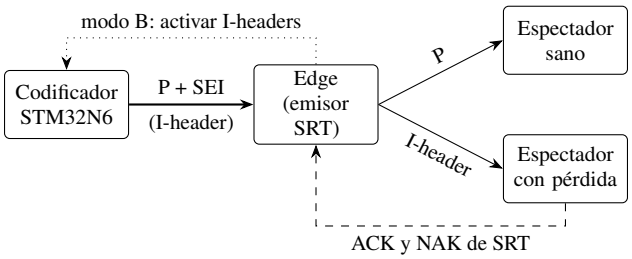


Fig. 1. Arquitectura. El I-header viaja siempre hasta el edge (modo A) y solo baja al espectador que perdió paquetes. El modo B, en el que el edge pide activar los I-headers, queda como trabajo futuro.

IV. Problema: ¿hay *macroblocking* con SRT?

Sea t_0 el instante en que el edge entrega un paquete a SRT, d el retardo de ida y D la duración de un corte de la red. Un paquete perdido llega a tiempo solo si su retransmisión arriba antes del plazo $t_0 + d + L$. El receptor no puede pedir nada hasta que llega un paquete posterior al corte, así que, en primera aproximación, la recuperación exige

$$D + \text{RTT} + d \lesssim L. \tag{1}$$

Por ejemplo, con $L = 120$ ms, $d = 15$ ms y RTT $= 30$ ms, SRT solo recupera cortes de hasta unos 75 ms. Si (1) no se cumple, hay dos casos. Con TLPKTDROP activo, que es lo obligatorio en modo live [1], el paquete se descarta. El TS llega con un hueco, el slice se trunca y el decodificador disimula lo que falta, normalmente copiando del frame anterior. Ese error se propaga con el movimiento hasta el próximo intra: es el *macroblocking*. Sin TLPKTDROP, en cambio, la entrega se detiene y la latencia crece. Por lo tanto, SRT no elimina el problema: lo traslada a los cortes más largos que $L - \text{RTT} - d$. Para cubrirlos hay que subir L para todos los espectadores, o acortar el GoP, lo que también pagan todos. Además, el emisor no se entera del descarte.

El problema que abordamos es, entonces, acotar el daño de esos cortes largos en el tramo edge–espectador con tres condiciones: no agregar latencia, no cambiar lo que recibe el espectador sin pérdidas y no pedir nada al codificador por cada pérdida. Suponemos que el tramo codificador–edge no pierde paquetes.

V. Estrategia y arquitectura

La Fig. 1 muestra la arquitectura. El codificador (un STM32N6570-DK, con codificador H.264 por hardware) envía al edge el stream con GoP largo y, cada n P, un I-header dentro de una SEI (modo A). Hacia cada espectador, el edge abre una conexión SRT. Al espectador sano le quita las SEI, así que recibe exactamente el stream con GoP largo. Cuando detecta que un espectador descartó paquetes, reemplaza el próximo P que tenga I-header por la SEI recovery point y el I-header, solo para ese espectador.

A. Cómo sabe el edge que un espectador descartó

Todo paquete SRT, incluido el ACK, lleva una marca de tiempo en microsegundos relativa al inicio de la conexión [1]. Un ACK que se genera para un paquete que llegó a tiempo sale, a lo sumo,

10 ms después de su llegada, lo que en general es mucho antes de su plazo. En cambio, el ACK que cubre un paquete saltado se genera en su plazo o después de él. Sean g el instante de generación del ACK, estimado por el edge como su instante de llegada menos RTT/2; $\tau_s = t_{0,s} + d + L$ el plazo del paquete s, y r_s el instante de su última retransmisión. El edge marca s como descartado si

$$s \in \mathcal{N} \;\wedge\; g \geq \tau_s \;\wedge\; \neg\big(r_s + d \leq \tau_s \wedge g \leq r_s + d + T_{\mathrm{ACK}}\big), \quad (2)$$

donde $\mathcal{N}$ son los paquetes que el edge vio en algún NAK y $T_{\mathrm{ACK}} = 10\,\mathrm{ms}$. La última condición excluye un paquete recuperado justo antes de su plazo cuyo ACK salió en el tick siguiente. Sin ella, los paquetes que quedaban detrás de un hueco recuperado se marcaban como perdidos. Como alternativa (cota superior), el espectador podría avisar cada descarte con un mensaje nuevo, lo que sí cambiaría el protocolo.

B. Generación del I-header

En el STM32N6, la API (*Application Programming Interface*) del codificador permite elegir el tipo de cada frame (IDR, P o intra no-IDR), fijar su QP (parámetro de cuantización) y dividir la imagen en slices por filas de macrobloques. Como la SEI de usuario que arma el hardware no admite más de 2048 B, la SEI del I-header se arma por software. En el PC, libx264 vía FFmpeg no emite intras no-IDR sin cuadros B, así que el núcleo reescribe el encabezado del slice: cambia el tipo de NAL de 5 a 1, pone el `frame_num` del P que reemplaza, quita el `idr_pic_id`, ajusta la marca de referencias y corrige `slice_qp_delta` si los PPS (*Picture Parameter Set*) difieren en el QP inicial. El resto del slice se copia bit a bit. Además, el I-header se codifica con el mismo QP que el P al que reemplaza. En el PC, eso se emula generando intras a QP constante (de 23 a 50, de 3 en 3) y eligiendo, en cada frame, el de QP más cercano al del P, que el núcleo lee del encabezado del slice. Con un I-header de calidad fija, la deriva era grande justo donde el codificador estaba más ajustado de bits.

C. Emulador

El núcleo (C++17, sin memoria dinámica, sin excepciones y sin dependencias del sistema operativo) lee el bitstream H.264, envuelve y desenvuelve la SEI, reescribe el encabezado y genera el TS. Es la parte que correrá en el STM32N6, separada de la HAL (*Hardware Abstraction Layer*). En el PC, un simulador de eventos discretos modela SRT en modo live: NAK inmediato y periódico, retransmisión con prioridad, ACK cada 10 ms, TSBPD, TLPKTDROP con ACK falso y un ritmo de envío limitado. El canal es de Gilbert–Elliott: un estado malo sin entrega, de duración media D, y una fracción p del tiempo en ese estado, evaluados en ranuras de 1 ms. Como la pérdida depende solo del tiempo y de la semilla, todos los escenarios ven la misma historia de cortes. El espectador reconstruye cada frame: una NAL con bytes faltantes se trunca en el primer byte perdido si es un slice, o se descarta si no lo es. Si se pierde el comienzo del PES, se pierde el frame entero. Sin pérdida, el video decodificado es idéntico bit a bit al original.

D. Métricas

PSNR (*Peak Signal-to-Noise Ratio*). Para cada frame k, se calcula el error cuadrático medio (MSE, *Mean Squared Error*), con Y_k la luminancia de referencia y $\hat{Y}_k$ la decodificada, ambas de $W \times H$ píxeles de 8 bits:

$$\mathrm{MSE}_k = \frac{1}{WH} \sum_{i,j} \big(Y_k(i,j) - \hat{Y}_k(i,j)\big)^2, \qquad \mathrm{PSNR}_k = 10 \log_{10} \frac{255^2}{\mathrm{MSE}_k} \;\; [\mathrm{dB}]. \quad (3)$$

La "señal" es el valor pico (255) y el "ruido" es la diferencia píxel a píxel entre ambas imágenes. Usamos dos referencias. $\mathrm{PSNR}_{\mathrm{ref}}$ compara con el mismo stream decodificado sin pérdida, así que solo mide lo que agregan la pérdida, el ocultamiento y la deriva. $\mathrm{PSNR}_{\mathrm{fuente}}$ compara con el video crudo e incluye la compresión. Dos frames idénticos dan $\mathrm{MSE} = 0$; ese caso se registra como 99 dB. Un frame cuenta como dañado si $\mathrm{PSNR}_{\mathrm{ref}} < 35$ dB, un valor elegido por nosotros para separar el artefacto visible de la deriva leve del I-header (en torno a 40 dB). Un frame que no llega se reemplaza por el anterior (congelamiento) antes de medir. Solo se usa la luminancia.

VMAF, de Netflix, también compara contra una referencia, pero no mide ruido: estima la calidad percibida en una escala de 0 a 100. Calcula varias medidas elementales (fidelidad de información visual en cuatro escalas, pérdida de detalle y movimiento de la referencia) y las combina con una regresión entrenada con notas de observadores. La instalación de FFmpeg usada no incluye VMAF, así que todavía no lo medimos. Las cifras de VMAF de este informe son las publicadas en [2, 4, 7].

VI. Resultados

Configuración. Los primeros 300 frames (10 s) de un clip de 1920×1080, reducidos a 1280×720 y tratados a 30 fps. Se codificaron con libx264 en perfil Baseline, a 3 Mbps con VBV (*Video Buffering Verifier*) de 1 s, una referencia y 45 slices (uno por fila de macrobloques), con GoP 90 y GoP 30. Los I-headers, uno cada 5 P, tienen el QP del P que reemplazan (Sección B). El retardo de ida edge–espectador es de 15 ms, la latencia de SRT de 120 ms y, sin SRT (UDP), hay un búfer de 20 ms. Usamos 8 semillas por régimen. Con UDP+I-header el espectador avisa cada descarte. Con SRT+I-header el edge usa la regla (2).

Deriva y costo del I-header. Para medir la deriva sola, se forzó un I-header sin pérdida en 10 frames distintos y se midió hasta el IDR siguiente (Tabla I). Donde el codificador trabaja con QP cercano a 29, el $\mathrm{PSNR}_{\mathrm{ref}}$ no baja de 39,3 dB. Donde está ajustado de bits (QP de 39 a 44, por mucho movimiento), el I-header y el P son dos aproximaciones gruesas y distintas de la misma imagen, y el $\mathrm{PSNR}_{\mathrm{ref}}$ cae a 28–35 dB. Sin embargo, contra la fuente la pérdida promedio es de alrededor de 1 dB en ambos casos. Es decir, en ese régimen el espectador ve una imagen distinta de la referencia, pero no mucho peor. El $\mathrm{PSNR}_{\mathrm{ref}}$ no distingue esto de un artefacto; una métrica perceptual como VMAF sí debería hacerlo. Enviar un I-header cada 5 P sube el tráfico codificador–edge de 3,1 a 5,2 Mbps (+68 %). Ese costo se paga solo en la subida, no en la conexión de cada espectador.

Tabla I. Deriva de un I-header forzado sin pérdida, medida hasta el IDR siguiente, según el QP del codificador en esa parte del clip.

Parte del clip	QP del P	min $\mathrm{PSNR}_{\mathrm{ref}}$	$\Delta\mathrm{PSNR}_{\mathrm{fuente}}$
Poco movimiento (4 casos)	≈ 29	39,3–45,0 dB	−1,4 dB
Mucho movimiento (6 casos)	39–44	28,0–35,2 dB	−1,0 dB

Tabla II. Frames dañados en 10 s (promedio de 8 semillas), con 2 % de pérdida (corte medio de 20 ms) y 4 % de pérdida (cortes de 200 y 600 ms). GoP 90 y $L = 120$ ms salvo que se indique. Las dos últimas columnas son del régimen de 200 ms: PSNR promedio contra la fuente (sin pérdida: 36,37 dB con GoP 90) y tasa hacia el espectador, incluidas las retransmisiones.

	Frames dañados, $D =$			$D = 200$ ms	
Escenario	20 ms	200 ms	600 ms	$\mathrm{PSNR}_{\mathrm{fuente}}$ (dB)	Tasa (kbps)
UDP	153,9	32,4	12,1	34,86	3279
UDP + I-header	76,6	9,8	5,9	35,91	3302
SRT	5,2	16,9	12,1	36,01	3321
SRT + I-header	15,8	8,1	5,8	36,13	3331
SRT + I-h., aviso	14,5	8,1	5,8	36,13	3331
SRT, GoP 30	6,9	10,5	8,4	35,99	3378
SRT, $L = 250$ ms	0,0	0,0	9,9	36,37	3346
SRT, $L = 500$ ms	0,0	0,0	3,2	36,37	3346
SRT, $L = 1000$ ms	0,0	0,0	0,0	36,37	3346

Comparación. La Tabla II resume los tres regímenes. Sin retransmisión (UDP), el I-header reduce el daño a la mitad o menos en los tres regímenes: de 153,9 a 76,6, de 32,4 a 9,8 y de 12,1 a 5,9 frames.

Con cortes de 20 ms, SRT con $L = 120$ ms recupera casi todo (5,2 frames dañados) y el I-header empeora la métrica (15,8). Los pocos descartes que quedan dañan poco, y el I-header agrega su deriva, que en la parte de mucho movimiento queda bajo el umbral (Tabla I). Contra la fuente, la diferencia es de solo 0,09 dB.

Con cortes de 200 y 600 ms, en cambio, SRT+I-header reduce el daño en un 52 % frente a SRT (de 16,9 a 8,1 y de 12,1 a 5,8 frames), más que GoP 30 (10,5 y 8,4), y además sube el PSNR contra la fuente. La tasa hacia el espectador crece menos de 1 % (unos 10 kbps), porque el I-header solo baja cuando hay pérdida. GoP 30, a la misma tasa, baja la calidad de todos los espectadores: 36,23 contra 36,37 dB sin pérdida. La regla (2) da los mismos resultados que el aviso explícito con cortes largos, y casi los mismos con cortes cortos (15,8 contra 14,5 frames).

La Fig. 2 muestra un corte. Con SRT solo, 48 frames quedan bajo el umbral y la imagen exacta vuelve recién con el IDR. Con GoP 30 son 27 frames, hasta el intra siguiente. Con I-header son 16: el daño se corta en el frame 230, unos 0,5 s después del inicio del corte, porque el edge debe esperar a que termine el corte y venza el plazo de los paquetes. Desde ahí la imagen queda en torno a 38 dB por la deriva, hasta el IDR. La Fig. 3 muestra la alternativa de subir L. Con $L = 250$ ms desaparece el daño de los cortes de 200 ms, pero los de 600 ms necesitan $L = 1000$ ms, y esa latencia la pagan todos los espectadores.

VII. Conclusiones y trabajo futuro

SRT no elimina el *macroblocking*: lo limita a los cortes más largos que su latencia menos el RTT y el retardo de ida, condición (1). Con $L = 120$ ms, cortes de 200 a 600 ms dejan entre 12 y 17 frames dañados cada 10 s, y con cortes de 600 ms

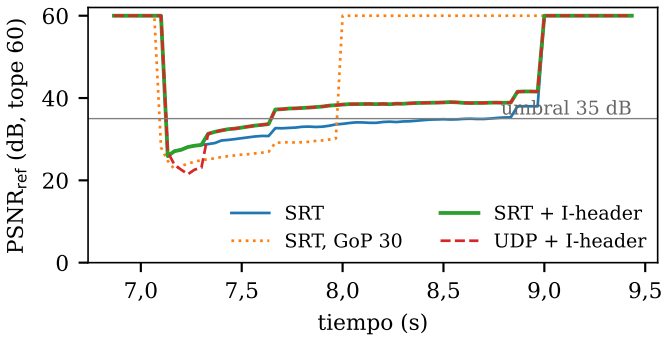


Fig. 2. $\mathrm{PSNR}_{\mathrm{ref}}$ por frame para el mismo corte (4 % de pérdida, cortes de 200 ms, semilla 1). Sin I-header, el daño dura hasta cerca del IDR siguiente. Con I-header, se corta en el primer I-header posterior a la detección y queda la deriva. Las curvas con I-header sobre SRT y sobre UDP coinciden. La línea gris es el umbral de daño.

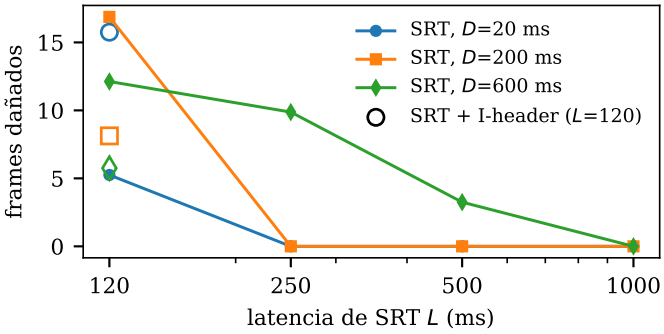


Fig. 3. Frames dañados (promedio de 8 semillas) para SRT solo según su latencia L, frente a SRT+I-header con $L = 120$ ms, por duración media del corte D.

SRT no mejora a UDP. SRT y el I-header se complementan: SRT recupera los cortes cortos y el I-header acota los largos, con un 52 % menos de frames dañados, sin agregar latencia y con menos de 1 % más de tasa hacia el espectador. Quien no pierde paquetes recibe exactamente el stream con GoP largo. Además, el edge puede saber qué descartó el espectador sin cambiar SRT, usando la marca de tiempo de los ACK.

Hay dos límites. El I-header llega recién después de la detección, unos $D + L + d$ tras el inicio del corte, y además hay que esperar hasta n frames al siguiente I-header. Y su deriva depende del QP: donde el codificador está ajustado de bits, el $\mathrm{PSNR}_{\mathrm{ref}}$ la confunde con daño, aunque contra la fuente cueste alrededor de 1 dB.

Como trabajo futuro quedan: (i) medir VMAF, para separar deriva de artefacto; (ii) una política del edge que no envíe el I-header cuando el QP es alto, el IDR está cerca o la pérdida es leve, y que detecte antes el descarte cuando el propio edge suprime una retransmisión que ya no llegaría a tiempo; (iii) barrer cada cuántos P va el I-header y evaluar el modo B; (iv) validar el modelo contra libsrt sobre una red emulada, con varios espectadores y pérdida en el tramo codificador–edge; (v) llevar el núcleo al STM32N6, con el `frame_num` y el QP del I-header fijados desde la API del codificador.

Referencias

[1] M. P. Sharabayko, M. A. Sharabayko, J. Dube, J. S. Kim y J. W. Kim, "The SRT protocol," IETF Internet-Draft draft-sharabayko-srt-01,

2021. [En línea]. Disponible: https://datatracker.ietf.org/doc/draft-sharabayko-srt/

[2] C. Chen, W. Yin, Z. Huang y S. Shi, "AGiLE: Enhancing adaptive GOP in live video streaming," en *Proc. ACM Multimedia Systems Conf. (MMSys '24)*, 2024, pp. 34–44, doi: 10.1145/3625468.3647609.

[3] ITU-T, "Advanced video coding for generic audiovisual services," Recomendación ITU-T H.264, ago. 2024. [En línea]. Disponible: https://www.itu.int/rec/T-REC-H.264

[4] G. Van Wallendael, H. Mareen, J. Vounckx y P. Lambert, "Keyframe insertion: Enabling low-latency random access and packet loss repair," *Electronics*, vol. 10, no. 6, art. 748, 2021, doi: 10.3390/electronics10060748.

[5] H. Mareen, M. Courteaux, P.-J. Speelmans, P. Lambert y G. Van Wallendael, "A study on keyframe injection in three generations of video coding standards for fast channel switching and packet-loss repair," *Multimedia Tools and Applications*, vol. 83, pp. 44485–44501, 2024, doi: 10.1007/s11042-023-17163-2.

[6] H. Mareen, P. Lambert y G. Van Wallendael, "P-frame injection for efficient packet-loss repair in ultra-low-latency video streaming," en *Proc. IEEE Int. Conf. Visual Communications and Image Processing (VCIP)*, 2023. [En línea]. Disponible: https://ieeexplore.ieee.org/document/10402720/

[7] B. Denizer, L. Winkel y O. Landsiedel, "FrameTrace: Frame-level telemetry for Media over QUIC," en *Proc. EMS '25, ACM SIGCOMM 2025 Workshops*, 2025, pp. 31–36, doi: 10.1145/3746441.3748231.